Chapter 6

Multiple linear regression

Exercises

Contents

6 Multiple linear regression

Exercises **1**
6.1 Nitrate concentration 3
6.2 Multiple linear regression model 9
6.3 MLR simulation exercise 17

6.1 Nitrate concentration

|||| Exercise 6.1 Nitrate concentration

In order to analyze the effect of reducing nitrate loading in a Danish fjord, it was decided to formulate a linear model that describes the nitrate concentration in the fjord as a function of nitrate loading. It was further decided to correct for fresh water runoff. The resulting model was

$$Y_i = \beta_0 + \beta_1 x_{1,i} + \beta_2 x_{2,i} + \varepsilon_i, \quad \varepsilon_i \sim N(0, \sigma^2), \tag{6-1}$$

where Y_i is the natural logarithm of nitrate concentration, $x_{1,i}$ is the natural logarithm of nitrate loading, and $x_{2,i}$ is the natural logarithm of fresh water runoff.

The parameters in the model were estimated in Python and the following results are available (here the output from `summary` has been slightly modified to conceal some of the output values):

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.3438
Model:                            OLS   Adj. R-squared:                  0.3382
No. Observations:                 240   F-statistic:                      62.07
Covariance Type:            nonrobust   Prob (F-statistic):            2.2e-16
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept     -2.36500      0.222    -10.661      0.000          *          *
x1              0.4762      0.062      7.720      0.000          *          *
x2              0.0827      0.070      1.185      0.237          *          *
==============================================================================
```

a) What are the estimates and standard errors for the model parameters ($\hat{\beta}_i$ and $\hat{\sigma}_{\beta_i}$)?

|||| Solution

The parameter estimates are given in the first column (`coef`) of the summary table, i.e.

$$\hat{\beta}_0 = -2.365$$
$$\hat{\beta}_1 = 0.476$$
$$\hat{\beta}_2 = 0.083$$

and the estimated standard errors for the parameters are given in the second column (`std err`):

$$\hat{\sigma}_{\beta_0} = 0.222$$
$$\hat{\sigma}_{\beta_1} = 0.062$$
$$\hat{\sigma}_{\beta_2} = 0.070$$

The estimated variances of the parameter estimates are the squares of these, e.g. $\hat{\sigma}^2_{\beta_0} = 0.222^2 = 0.0493$.

b) How many degrees of freedom are there in the estimation?

|||| Solution

The number of degrees of freedom is equal to $n - (p + 1)$, and since the number of observations is $n = 240$ (`No. Observations`) and there are $p = 2$ explanatory variables, we get

$$df = 240 - (2 + 1) = 237$$

c) Calculate the 95% confidence intervals for the parameters (β_0, β_1, and β_2).

Use Python to compute quantiles and probabilities from relevant probability distribution(s).

|||| Solution

From Method 6.5 we know that the confidence intervals can be calculated by

$$\hat{\beta}_i \pm t_{1-\alpha/2} \cdot \hat{\sigma}_{\beta_i},$$

where $t_{1-\alpha/2}$ is the $(1 - \alpha/2)$-quantile of a t-distribution with $n - (p + 1) = 237$ degrees of freedom. To use this formula we need three quantities for each parameter: the estimate $\hat{\beta}_i$, its standard error $\hat{\sigma}_{\beta_i}$ (both found in the previous question) and the t-quantile.

The t-quantile: with $\alpha = 0.05$ we need the 0.975 quantile of a t-distribution with 237 degrees of freedom:

```
print(stats.t.ppf(0.975, df=237))
1.970
```

Putting it all together: inserting the values in the formula gives

$$\begin{aligned}
\beta_0: &\quad -2.365 \pm 1.970 \cdot 0.222 = -2.365 \pm 0.437 = [-2.802;\ -1.928] \\
\beta_1: &\quad 0.476 \pm 1.970 \cdot 0.062 = \quad 0.476 \pm 0.122 = [0.354;\ 0.598] \\
\beta_2: &\quad 0.083 \pm 1.970 \cdot 0.070 = \quad 0.083 \pm 0.138 = [-0.055;\ 0.221]
\end{aligned}$$

These are the confidence intervals that would normally appear in the output table shown above if the values had not been modified to conceal them.

d) On level $\alpha = 0.05$ which of the parameters are significantly different from 0? What are the null hypothesis and the p-values of the corresponding hypothesis tests for each parameter?

|||| Solution

For each parameter we test the null hypothesis (Method 6.4)

$$H_{0,i}: \beta_i = 0 \quad \text{vs.} \quad H_{1,i}: \beta_i \neq 0.$$

We can see directly from the confidence intervals above that β_0 and β_1 are significantly different from zero (the confidence intervals do not cover zero), while we cannot reject that $\beta_2 = 0$ (the confidence interval covers zero).

The p-values we can see directly in the Python output (column `P>|t|`):

- for β_0 the p-value is less than 0.0005 (Python output is rounded to 0.000),
- for β_1 the p-value is less than 0.0005 (Python output is rounded to 0.000),
- for β_2 the p-value is 0.237.

So there is very strong evidence against the null hypothesis for β_0 and β_1, while the p-value for β_2 is larger than $\alpha = 0.05$.

The p-values can also be calculated from the test statistics in the column `t`, e.g. for β_2 the test statistic is $t_{\text{obs}} = \hat{\beta}_2 / \hat{\sigma}_{\hat{\beta}_2} = 1.185$, and we can use the formula:

$$p\text{-value} = 2 \cdot P(T > |t_{\text{obs}}|),$$

where T follows a t-distribution with 237 degrees of freedom:

```
print(2 * (1 - stats.t.cdf(1.185, df=237)))
0.2372
```

e) In the model, all the variables are the natural logarithms of the measured values (nitrate concentration, nitrate loading and fresh water runoff), i.e., they are all log-transformed variables and not the measured values themselves. Why do you think the log-transformed variables are used?

|||| Solution

There can be several reasons, and they are related to the model assumptions and to how the variables are expected to depend on each other. The consideration most relevant for our course is:

The measured values are positive and skewed: concentrations, loadings and runoffs can only be positive, and such quantities typically have right-skewed distributions (many small values and a few very large ones). Taking the logarithm makes the values more symmetric and pulls in the very large values, so that a few extreme observations do not dominate the fit.

Whether the log transformation actually makes the model assumptions hold should be checked with a residual analysis (q-q plot and residuals vs. fitted values), which requires the data and not only the summary table.

Also, predictions are made on the log scale, and when they are transformed back with the exponential function they are always positive, as a concentration must be.

More advanced considerations:

The variance is often proportional to the level: for this kind of data the variation is typically larger when the values are large (e.g. $\pm 10\%$ rather than $\pm$ a fixed amount). On the original scale this would violate the assumption that ε_i has constant variance σ^2, whereas on the log scale a constant *relative* variation becomes a constant variance.

The relationship is likely multiplicative: it is reasonable that the concentration changes *proportionally* with the loading, i.e. a model of the form

$$\text{concentration} = e^{\beta_0} \cdot \text{loading}^{\beta_1} \cdot \text{runoff}^{\beta_2} \cdot e^{\varepsilon_i}.$$

Taking the natural logarithm on both sides gives exactly the linear model above. In this model β_1 can be interpreted in relative terms: if the nitrate loading is doubled (and the runoff is kept fixed), the nitrate concentration is multiplied by $2^{\hat{\beta}_1} = 2^{0.476} = 1.39$, i.e. it increases by approximately 39%.

Exam-style questions

f) Related to the regression model above:

$$Y_i = \beta_0 + \beta_1 x_{1,i} + \beta_2 x_{2,i} + \varepsilon_i, \quad \varepsilon_i \sim N(0, \sigma^2) \tag{6-2}$$

Which of the following assumptions are required for the standard multiple linear regression model? (only one option is true)

1) $\varepsilon_i = 0$ for all $i = 1, \ldots, n$, and β_j follows a normal distribution
2) $E[x_1] = E[x_2] = 0$ and $V[\varepsilon_i] = \beta_1^2$
3) $E[\varepsilon_i] = 0$ and $V[\varepsilon_i] = \beta_1^2$
4) ε_i is normally distributed with constant variance, and ε_i and ε_j are independent for $i \neq j$
5) $\varepsilon_i = 0$ for all $i = 1, \ldots, n$, and x_j follows a normal distribution for $j = 1, 2$

|||| Solution

1) ε_i follows a normal distribution with expectation equal to zero, but the realizations are not zero. Further, β_j is deterministic and hence it does not follow a distribution ($\hat{\beta}_j$ does), hence 1) is not correct.

2)–3) There are no assumptions on the expectation of x_j, and the variance of ε_i equals σ^2, not β_1^2, hence 2) and 3) are not correct.

4) Is correct, this is the usual assumption about the errors.

5) Is incorrect since ε_i follows a normal distribution (the realizations are not zero). Further, there are no distributional assumptions on x_j. In fact we assume that x_j is known.

So the correct answer is **4)**.

g) In the nitrate model above, $\hat{\beta}_2 = 0.083$ is not significantly different from 0 ($p = 0.237$). Which of the following is the most careful, correct conclusion?

1) Since the p-value is larger than 0.05, we have shown that $\beta_2 = 0$, i.e. fresh water runoff has no effect on the nitrate concentration.

2) Since $\hat{\beta}_2 > 0$, more fresh water runoff increases the nitrate concentration, also when the nitrate loading is kept fixed.
3) When nitrate loading is in the model, the data do not show that fresh water runoff explains additional variation in the nitrate concentration.
4) Since not all parameters are significant, the model cannot be used, and therefore the estimate $\hat{\beta}_1$ cannot be used either.
5) The probability that $\beta_2 = 0$ is 0.237, so it is most likely that fresh water runoff has an effect on the nitrate concentration.

|||| Solution

Correct answer: **3)**. In multiple linear regression, the test of $H_0 : \beta_2 = 0$ concerns the effect of x_2 *when the other variables (x_1) are in the model*. Not rejecting H_0 means that the data do not provide evidence that x_2 explains variation in Y in addition to what x_1 already explains.

The other statements are wrong:

1) Not rejecting H_0 is not the same as showing that H_0 is true: the 95% confidence interval $[-0.055;\ 0.221]$ contains both zero and positive values, so we simply do not know (the same point as for hypothesis tests in Chapter 3).
2) The estimate is positive, but the effect is not significant, so we cannot conclude that the true β_2 is positive.
4) A non-significant parameter does not make the model useless. The usual procedure is to remove x_2 and refit the model (model reduction), and β_1 is clearly significant.
5) The p-value is not the probability that H_0 is true. It is the probability of observing a test statistic at least as extreme as 1.185 *given* that $\beta_2 = 0$.

6.2 Multiple linear regression model

Exercise 6.2 Multiple linear regression model

The following measurements have been obtained in a study:

No.	1	2	3	4	5	6	7	8	9	10	11	12	13
y	1.45	1.93	0.81	0.61	1.55	0.95	0.45	1.14	0.74	0.98	1.41	0.81	0.89
x_1	0.58	0.86	0.29	0.20	0.56	0.28	0.08	0.41	0.22	0.35	0.59	0.22	0.26
x_2	0.71	0.13	0.79	0.20	0.56	0.92	0.01	0.60	0.70	0.73	0.13	0.96	0.27

No.	14	15	16	17	18	19	20	21	22	23	24	25
y	0.68	1.39	1.53	0.91	1.49	1.38	1.73	1.11	1.68	0.66	0.69	1.98
x_1	0.12	0.65	0.70	0.30	0.70	0.39	0.72	0.45	0.81	0.04	0.20	0.95
x_2	0.21	0.88	0.30	0.15	0.09	0.17	0.25	0.30	0.32	0.82	0.98	0.00

The data can be stored in a Python DataFrame named `df` by copy-pasting the following code:

```
df = pd.DataFrame({
    'x1': [0.58, 0.86, 0.29, 0.20, 0.56, 0.28, 0.08, 0.41,
       0.22, 0.35, 0.59, 0.22, 0.26, 0.12, 0.65, 0.70, 0.30,
       0.70, 0.39, 0.72, 0.45, 0.81, 0.04, 0.20, 0.95],
    'x2': [0.71, 0.13, 0.79, 0.20, 0.56, 0.92, 0.01, 0.60,
       0.70, 0.73, 0.13, 0.96, 0.27, 0.21, 0.88, 0.30, 0.15,
       0.09, 0.17, 0.25, 0.30, 0.32, 0.82, 0.98, 0.00],
    'y': [1.45, 1.93, 0.81, 0.61, 1.55, 0.95, 0.45, 1.14,
       0.74, 0.98, 1.41, 0.81, 0.89, 0.68, 1.39, 1.53, 0.91,
       1.49, 1.38, 1.73, 1.11, 1.68, 0.66, 0.69, 1.98]
})
```

It is expected that the response variable y can be described by the independent variables x_1 and x_2. This implies that the parameters of the following model should be estimated and tested:

$$Y_i = \beta_0 + \beta_1 x_{1,i} + \beta_2 x_{2,i} + \varepsilon_i, \quad \varepsilon_i \sim N(0, \sigma^2).$$

a) Use Python to fit the model and compute parameter estimates ($\hat{\beta}_0$, $\hat{\beta}_1$, $\hat{\beta}_2$, and $\hat{\sigma}^2$). Also find the 95% confidence intervals for β_0, β_1, and β_2.

Solution

The question is answered by Python. We start by estimating the parameters:

```
fit = smf.ols('y ~ x1 + x2', data=df).fit()
print(fit.summary(slim=True))

                            OLS Regression Results
==============================================================================
Dep. Variable:                          y   R-squared:                       0.940
Model:                                OLS   Adj. R-squared:                  0.934
No. Observations:                      25   F-statistic:                     172.0
Covariance Type:                nonrobust   Prob (F-statistic):           3.70e-14
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept      0.4335      0.066      6.571      0.000       0.297       0.570
x1             1.6530      0.095     17.355      0.000       1.455       1.851
x2             0.0039      0.075      0.053      0.958      -0.151       0.159
==============================================================================
```

The parameter estimates are given in the first column (`coef`), i.e.

$$\hat{\beta}_0 = 0.434$$
$$\hat{\beta}_1 = 1.653$$
$$\hat{\beta}_2 = 0.0039$$

The error variance: the residual standard deviation $\hat{\sigma}$ is found as the square root of the residual variance stored in the fitted model (`fit.scale`):

```
# Residual standard deviation
sigma = np.sqrt(fit.scale)
print(sigma)
0.1127
```

so the error variance estimate is $\hat{\sigma}^2 = 0.1127^2 = 0.0127$.

The confidence intervals are given in the last two columns (`0.025` and `0.975`), i.e.

$$\beta_0 : [0.297;\ 0.570]$$
$$\beta_1 : [1.455;\ 1.851]$$
$$\beta_2 : [-0.151;\ 0.159]$$

b) Can the model be reduced? Use backward selection (Section 6.3 in the book) with significance level $\alpha = 0.05$:

– Which explanatory variable has the weakest evidence of an effect on y? Is the effect significant at $\alpha = 0.05$?

– If appropriate, remove the variable and fit the reduced model. Are the remaining parameters significant? Compare the estimate of β_1 with the one from the full model.

|||| Solution

In backward selection we start with the full model and remove the least significant explanatory variable, one at a time, as long as it is not significant.

The weakest variable: in part a) of this exercise we can see that x_2 has by far the largest p-value (0.958), it being much larger than $\alpha = 0.05$. Moreover, we can also observe how it's 95% confidence interval $[-0.151;\ 0.159]$ contains zero, so we cannot reject that $H_0 : \beta_2 = 0$; and we conclude that there is no evidence saying that x_2 has an effect on y (when x_1 is in the model).

The reduced model: Given the conclusions above x_2 is removed from the model to get the simplified model

$$Y_i = \beta_0 + \beta_1 x_{1,i} + \varepsilon_i, \quad \varepsilon_i \sim N(0, \sigma^2).$$

Now, we can get the parameter estimates in the simpler model in python

```
fit = smf.ols('y ~ x1', data=df).fit()
print(fit.summary(slim=True))
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                          y   R-squared:                       0.940
Model:                                OLS   Adj. R-squared:                  0.937
No. Observations:                      25   F-statistic:                     359.6
Covariance Type:                nonrobust   Prob (F-statistic):           1.54e-15
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept      0.4361      0.044      9.913      0.000       0.345       0.527
x1             1.6512      0.087     18.963      0.000       1.471       1.831
==============================================================================
```

and we can see how both remaining parameters are significant (their p-values are less than 0.05 and neither confidence interval covers zero), so the model should not be reduced further.

Comparing $\hat{\beta}_1$: If we check the estimate of β_1 in this new simplified model, we can check that it has hardly changed from the full model (1.653 vs. 1.651), and the model's R^2 is still 0.940 as in the full model, which means that x_2 did not explain any extra variation in y.

c) Carry out a residual analysis to check that the model assumptions are fulfilled (for the reduced model).

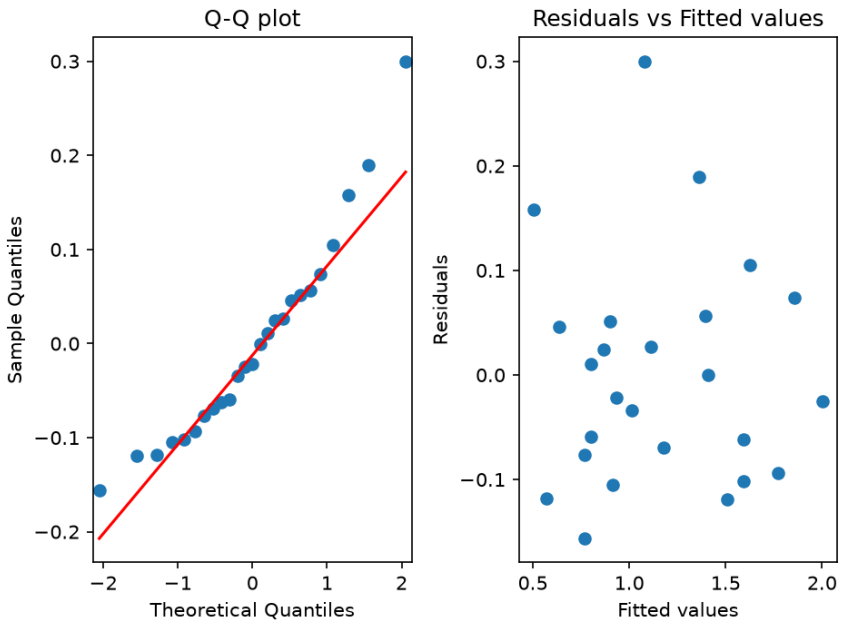
Solution

We are interested in inspecting two plots of the residuals ($e_i = y_i - \hat{y}_i$) from the reduced model:

- A q-q plot of the residuals: if the errors are normally distributed, the points should lie approximately on a straight line,
- A plot of the residuals versus the fitted values $\hat{y}_i$: there should be no pattern, and the spread of the residuals should be the same for all fitted values (constant variance).

The residuals and the fitted values are stored in the fitted model as `fit.resid` and `fit.fittedvalues`:

```
# Prepare plot
fig, ax = plt.subplots(1,2)
# Q-Q plot
sm.qqplot(fit.resid, line="q", a=1/2, ax=ax[0])
ax[0].set_title("Q-Q plot")
# Scatter plot of residuals vs. fitted values
ax[1].scatter(fit.fittedvalues, fit.resid)
ax[1].set_xlabel("Fitted values")
ax[1].set_ylabel("Residuals")
ax[1].set_title("Residuals vs Fitted values")
plt.tight_layout()
plt.show()
```



There is no strong evidence against the assumptions: the points in the q-q plot lie approximately on a straight line, and there is no obvious dependence between the residuals and the fitted values. We conclude that we have no reason to suspect a violation of the assumptions.

d) Make a plot of the fitted line and 95% confidence and prediction intervals of the line for $x_1 \in [0, 1]$ (for the reduced model).

Solution

The intervals for the line are found with Python (Method 6.9). We do it in three steps.

New values of x_1: we make a DataFrame with 100 values of x_1 evenly spread between 0 and 1:

```
x1_new = pd.DataFrame({'x1': np.linspace(0, 1, 100)})
```

The intervals: for each new value of x_1 we compute the point on the line and the 95% confidence and prediction intervals (here we print the first 5 rows):

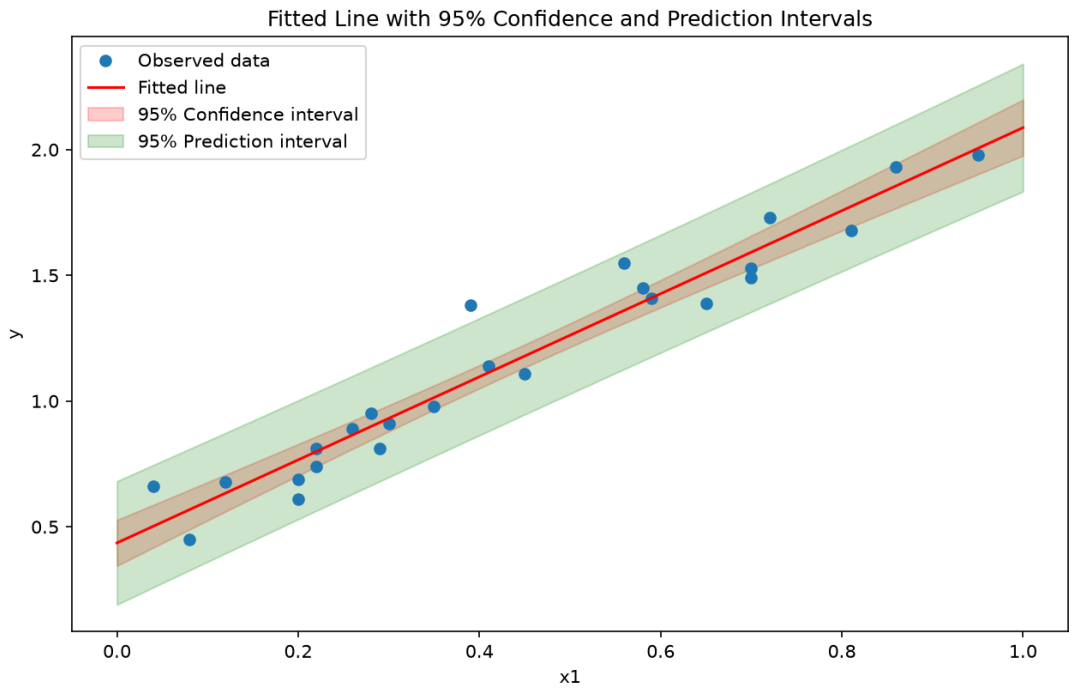
```
prediction_summary = fit.get_prediction(x1_new).summary_frame(alpha=0.05)
print(prediction_summary.head())

       mean    mean_se  mean_ci_lower  mean_ci_upper  obs_ci_lower  obs_ci_upper
0  0.436091  0.043993       0.345085       0.527098      0.190541      0.681641
1  0.452770  0.043234       0.363334       0.542207      0.207798      0.697743
2  0.469449  0.042480       0.381573       0.557326      0.225042      0.713857
3  0.486128  0.041731       0.399801       0.572455      0.242274      0.729982
4  0.502807  0.040987       0.418020       0.587594      0.259493      0.746121
```

Here "mean" is the point **on** the fitted line, "mean_ci_lower" and "mean_ci_upper" are the limits of the **confidence interval** for the line, and "obs_ci_lower" and "obs_ci_upper" are the limits of the **prediction interval**.

The plot:
(you do not have to use "plt.fill_between", it is also fine to simply plot the intervals using lines)

```
plt.figure(figsize=(10, 6))
plt.plot(df['x1'], df['y'], 'o', label='Observed data')
plt.plot(x1_new, prediction_summary['mean'], 'r-', label='
   Fitted line')
plt.fill_between(x1_new['x1'],
                 prediction_summary['mean_ci_lower'],
                 prediction_summary['mean_ci_upper'],
                 color='red', alpha=0.2, label='95%
                    Confidence interval')
plt.fill_between(x1_new['x1'],
                 prediction_summary['obs_ci_lower'],
                 prediction_summary['obs_ci_upper'],
                 color='green', alpha=0.2, label='95%
                    Prediction interval')
plt.xlabel('x1')
plt.ylabel('y')
plt.legend()
plt.title('Fitted Line with 95% Confidence and Prediction
   Intervals')
plt.show()
```



The red band (confidence interval) shows where we expect the true line to be, while the green band (prediction interval) shows where we expect 95% of new observations to fall. The prediction interval is wider, since it also includes the variation of a single new observation around the line. Both bands are narrowest near the mean $\bar{x}_1 = 0.437$ and become wider towards the ends of the interval.

Exam-style questions

e) This question regards the 95% confidence interval and the 95% prediction interval for a linear regression model with explanatory variable x. Which of the following statements is false?

1) The prediction interval is always at least as wide as the confidence interval, at every value of x.
2) Both intervals are narrowest near $\bar{x}$ and widen toward the edges of the plotted range.
3) The vertical distance between the prediction interval's upper and lower curves stays constant across all x, unlike the confidence interval.
4) The confidence interval describes uncertainty about the *mean* response at each x, not about any individual future observation.

5) Extending the plot to x values well outside the observed range of x-values would make both intervals progressively less trustworthy.

Solution

Correct answer: **3)**. Both intervals share the same $\sqrt{\cdot + \frac{(x_1 - \bar{x}_1)^2}{S_{xx}}}$ term, so the prediction interval also narrows near $\bar{x}_1$ and widens toward the edges — it is not constant width, just consistently *wider* than the confidence interval at every point (because of the extra "+1" term for individual-observation scatter). Statements 1), 2), 4), and 5) are all correct facts about confidence vs. prediction intervals, several of which echo points already established for simple regression in Chapter 5.

6.3 MLR simulation exercise

|||| Exercise 6.3 MLR simulation exercise

The following measurements have been obtained in a study:

Nr.	1	2	3	4	5	6	7	8
y	9.29	12.67	12.42	0.38	20.77	9.52	2.38	7.46
x_1	1.00	2.00	3.00	4.00	5.00	6.00	7.00	8.00
x_2	4.00	12.00	16.00	8.00	32.00	24.00	20.00	28.00

a) Plot the observed values of y as a function of x_1 and x_2. Does it seem reasonable that either x_1 or x_2 can describe the variation in y?

|||| Solution

Scatter plots of y vs. x_1 and y vs. x_2 show no clear linear relationship with either predictor individually.

b) Estimate the parameters for the two simple linear models

$$Y_i = \beta_0 + \beta_1 x_{1,i} + \varepsilon_i \quad \text{and} \quad Y_i = \beta_0 + \beta_1 x_{2,i} + \varepsilon_i,$$

and report 95% confidence intervals for all parameters. Are any parameters significantly different from zero at 5%?

|||| Solution

Fitting both models shows that all confidence intervals include zero, so we cannot reject that any of the parameters are zero. Neither x_1 nor x_2 alone explains the variation in y.

c) Estimate the parameters for the full multiple linear regression model

$$Y_i = \beta_0 + \beta_1 x_{1,i} + \beta_2 x_{2,i} + \varepsilon_i, \quad \varepsilon_i \sim N(0, \sigma^2),$$

and go through the steps of the MLR method (use confidence level 0.05).

```
y = np.array
    ([9.29,12.67,12.42,0.38,20.77,9.52,2.38,7.46])
x1 = np.array([1.,2.,3.,4.,5.,6.,7.,8.])
x2 = np.array([4.,12.,16.,8.,32.,24.,20.,28.])
D = pd.DataFrame({'y':y,'x1':x1,'x2':x2})
print(smf.ols('y ~ x1 + x2', data=D).fit().summary())
```

Solution

The MLR fit shows that all three parameters (β_0, β_1, β_2) are significant at 5%. Residual analysis (normal Q-Q plot and residuals vs. fitted values) reveals a quadratic pattern in the residuals as a function of x_1, suggesting that a x_1^2 term should be added. Including $x_3 = x_1^2$ in the model makes all parameters significant and removes the pattern, giving the finally selected model:

$$Y_i = \beta_0 + \beta_1 x_{1,i} + \beta_2 x_{2,i} + \beta_3 x_{1,i}^2 + \varepsilon_i.$$

d) Find the standard error for the line, and the confidence and prediction intervals for the points $(\min x_1, \min x_2)$, $(\bar{x}_1, \bar{x}_2)$, $(\max x_1, \max x_2)$.

Solution

Using the extended model with $x_3 = x_1^2$:

```
D['x3'] = D['x1']**2
fit = smf.ols('y ~ x1 + x2 + x3', data=D).fit()
Dn = pd.DataFrame({'x1':[1,4.5,8],'x2':[4,18,32],'x3'
    :[1,20.25,64]})
print(fit.get_prediction(Dn).summary_frame())
```

The standard error for the line is smallest at $(\bar{x}_1, \bar{x}_2)$ and largest at the extremes.

e) Plot the observed values together with the fitted values as a function of x_1.

Solution

When x_1, x_2, and x_1^2 are all included, the fitted values are very close to the observed values, giving an almost perfect fit. In contrast, neither x_1 nor x_2 alone could predict the outcomes.